ASTRA / MARCO 5 / #31

Excluir uma tarefa.

A confirmação acontece antes.
A lista muda depois da API.

ASTRA

`DELETE /tarefas/:id`

01 / NAVEGADOR
Confirma a intenção

02 / API
Remove o registro

03 / SVELTE
Atualiza a lista

Uma exclusão pode voltar sem JSON.

src/lib/api.ts

```
await fetch(`/tarefas/${id}`, {
  method: 'DELETE'
});

// conferir resposta.ok
// não chamar resposta.json()
```

RESPOSTA HTTP

204

No Content: sucesso,
sem corpo de resposta.

O ID é codificado antes de entrar no caminho da requisição.

Usamos o `fetch` do navegador e o json-server que já existiam. Nenhum pacote novo.

#31 / COMPONENTES E REATIVIDADE

Quem faz o quê na exclusão?

01 / CARTÃO	02 / API.TS	03 / CALLBACK	04 / SVELTE
Confirmar	Excluir	Avisar a página	Atualizar a lista

01

O cartão usa window.confirm. Cancelar encerra o fluxo sem enviar DELETE.

- `TarefaCard.svelte`

```
if (!window.confirm(...)) return;
```

O cartão representa uma tarefa. A página administra a coleção.

#31 / DEMONSTRAÇÃO LOCAL

A tarefa só sai depois do sucesso.

Preparar a apresentação

A fazer · prioridade média

Excluir ×

Esta demonstração não acessa a API real.

Cancelar mantém a tarefa e não envia requisição.

Confirmar aguarda a API. Durante o envio, os controles ficam bloqueados.

Falhar mantém a tarefa e mostra um aviso.

`onexcluir(id)` avisa a página; `$state` muda e `$derived` reaplica os filtros.

#32 / CARREGAMENTO

A busca começa no onMount.

src/routes/+page.svelte

```
let carregando = $state(true);

onMount(async () => {
  // buscar tarefas na API
});
```

PRÉVIA DA LISTA / SIMULAÇÃO LOCAL

- Tarefas recebidas

onMount roda quando o componente é montado no navegador.

A mensagem de espera evita confundir uma requisição em andamento com uma lista vazia.

#32 / ESTADO REATIVO NO SVELTE 5

Como a reatividade atualiza a tela.

`await` aguarda os dados sem bloquear a página inteira.

`finally` encerra a espera, mesmo quando a busca falha.

`$state` permite ao Svelte acompanhar os valores usados na interface.

Trecho do carregamento

```
try {
  tarefas = await listarTarefas();
} /* catch trata a falha */
finally {
  carregando = false;
}
```

Não escondemos o HTML manualmente. O bloco `{#if carregando}` reage ao novo valor.

#33 / TRATAMENTO DE ERRO NA LISTA

A falha precisa chegar até a tela.

api.ts / página

```
// a API já verificava:
if (!resposta.ok) throw ...;

// a página passa a tratar:
catch (e) {
  erro = 'Não foi possível...';
}
```

- Não foi possível carregar as tarefas.

`fetch` pode receber HTTP 500 sem rejeitar a Promise. Conferir `resposta.ok` é necessário.

A mensagem está implementada. O botão “Tentar novamente” ainda não está no código.

#32 + #33 / RENDERIZAÇÃO CONDICIONAL

Um estado, uma resposta visual.

```
{#if carregando}
Aguardando a resposta
{:else if erro}
A busca falhou
{:else if ...length === 0}
Nenhum resultado
{:else}
Exibir os cartões
```

SAÍDA DO TEMPLATE / SIMULAÇÃO LOCAL

- Carregando tarefas...

O primeiro bloco verdadeiro define o que aparece. O Svelte atualiza essa região quando o estado muda.

O estado vazio já aparece no código; esta demonstração não declara a issue #34 concluída.

#36 / FALHA AO SALVAR

O erro não apaga o formulário.

Título da tarefa

Preparar a apresentação

Simulação local: nenhum dado é enviado à API.

Os campos usam `bind:value`. O `catch` altera o aviso, sem limpar o estado do formulário.

TarefaFormulario.svelte

```
try {
  await aoEnviar({ ...campos });
} catch {
  erroEnvio = 'Não foi
possível...';
} finally {
  enviando = false;
}
```

Cadastro e edição reutilizam o formulário. Em falha, a pessoa continua na página com o texto preservado.

#36 / FALHA AO EXCLUIR

Sem sucesso na API, o cartão fica.

Preparar a apresentação

A tarefa permanece nesta simulação.

Simulação local: a API real não é acessada.

O aviso aparece e o botão fica disponível para repetir a operação.

TarefaCard.svelte

```
try {
  await excluirTarefa(tarefa.id);
  onexcluir(tarefa.id);
} catch {
  erroExclusao = 'Não foi
possível...';
} finally {
  excluindo = false;
}
```

Conferimos falhas HTTP no cadastro, na edição e na exclusão: aviso visível, dados mantidos e controles liberados.

#37 / ACESSIBILIDADE

Conferir a experiência além do mouse.

Lighthouse no DevTools ajudou a conferência. Teclado e foco visível entram na verificação manual.

Título da tarefa

Preparar a apresentação

Conferir tarefa

Demonstração local de campo e botão.

HTML no componente Svelte

```
<label for="titulo">Título</label>
<input id="titulo" />

/* CSS compartilhado */
:focus-visible {
  outline: 2px solid ...;
  outline-offset: 4px;
}
```

O Svelte gera controles HTML nativos. Label e foco continuam sendo recursos da plataforma.

Tab percorre os controles. Shift + Tab volta. Enter aciona o botão focado.

#37 / RESPONSIVIDADE E AUDITORIA

O mesmo quadro em telas diferentes.



Simulação dos quatro, dois e um espaços de coluna usados no Kanban.

1280 px · quatro colunas

quadro/+page.svelte · CSS

```
.quadro-container { /* 4 colunas */
}

@media (max-width: 1100px) {
  /* 2 colunas */
}
@media (max-width: 640px) {
  /* 1 coluna */
}
```

O navegador adapta o layout com media queries; não duplicamos páginas no Svelte.

Brenno · Lighthouse no DevTools · PR #71. A entrega também adicionou a descrição do site em `src/app.html`.